P2538R0
ADL-proof std::projected

Published Proposal, 2022-02-04

Author:
Arthur O'Dwyer

Audience:
LEWG, LWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Draft Revision:
2

Current Source:
github.com/Quuxplusone/draft/blob/gh-pages/d2538-adl-proof-std-projected.bs

Current:
rawgit.com/Quuxplusone/draft/gh-pages/d2538-adl-proof-std-projected.html

---

Abstract

When `it` is a pointer of type `Danger*`, evaluating `*it` does not perform ADL and does not attempt to complete the associated type `Danger`. But when `proj` is an iterator of type `projected<Danger*, identity>`, evaluating `*proj` does perform ADL and will attempt to complete `Danger`. As a result, most Ranges algorithms are fundamentally "less ADL-proof" than their STL Classic counterparts, and some concepts such as `indirectly_comparable` can result in unexpected hard errors. We fix this by respecifying `projected<I, Proj>` in a backward-compatible way so that its associated entities do not include its template arguments.

Table of Contents

1. Changelog

- R0:
  - Initial revision.

2. Background on ADL-proofing

Throughout this paper, we use `Holder<Incomplete>` as the canonical example of a "dangerous-to-complete" type.

```
template<class T> struct Holder { T t; };
struct Incomplete;
```

Any operation that requires type `Holder<Incomplete>` to be completed will trigger a hard error. For example:

```
error: field has incomplete type 'Incomplete'
template<class T> struct Holder { T t; };
                                  ^
```

```
<source>:6:20: note: in instantiation of template class 'Holder<Incomplete>
Holder<Incomplete> h;
                   ^
```

One such operation is ADL. For example, this snippet will trigger a hard error:

```
Holder<Incomplete> *p;
int f(Holder<Incomplete>*);
int x = f(p);  // error: ADL requires completing the associated type Holder
```

but this snippet will be OK:

```
Holder<Incomplete> *p;
int f(Holder<Incomplete>*);
int x = ::f(p);  // OK: no ADL, therefore no error
```

Most operations on native pointers do not trigger ADL. For example:

```
Holder<Incomplete> *a[10] = {}; // ten null pointers
Holder<Incomplete> **p = a;                  // OK
p += 1;                                      // OK
assert(*p == nullptr);                       // OK
assert(p == a+1);                            // OK
assert(std::count(a, a+10, nullptr) == 10); // OK
```

The libc++ test suite includes a lot of "ADL-proofing" test cases, to make sure that our STL algorithms don’t unnecessarily trigger the completion of associated types. As far as I know, we consider this _not_ a conformance issue, but a pretty important quality-of-implementation issue. Unnecessarily prematurely completing a type can cause hard errors for the user that are difficult to fix, and I imagine it could cause ODR issues too.

For more information, see [Uglification].

§ 3. The problem in C++20

Most C++20 Ranges algorithms fundamentally cannot be ADL-proofed.

```
Holder<Incomplete> *a[10] = {}; // ten null pointers
assert(std::count(a, a+10, nullptr) == 10); // OK
assert(std::ranges::count(a, a+10, nullptr) == 10); // hard error
```

This causes a hard error on all possible implementations. Because the error messages are so long, I've moved them into Appendix A.

In fact, even the following program causes hard errors:

```
using T = Holder<Incomplete>*;

static_assert(std::equality_comparable<T>); // OK
bool x = std::indirectly_comparable<T*, T*, std::equal_to<>>; // hard error
bool y = std::sortable<T*>; // hard error
```

The error happens because `indirectly_comparable<T*, T*, Pred>` is actually defined as `indirect_binary_predicate<Pred, projected<T*, identity>, projected<T*, identity>>`. In evaluating that concept, we eventually need to ask whether `*it` is a valid expression for an iterator of type `projected<T*, identity>`. This means we need to complete all the associated types of `projected<T*, identity>`, which includes all the associated types of `T`, which includes `Holder<Incomplete>`.

So, we are in the sad state that `std::sort`, `std::count`, etc., are safe to use on "dangerous-to-ADL" types, but their `std::ranges::` counterparts are not.

§ 4. The solution: ADL-proof `std::projected`

To fix the problem and ADL-proof all the Ranges algorithms, we simply have to stop `T` from being an associated type of `projected<T*, identity>`. We can do this by inserting an ADL firewall.

The basic technique is to replace

```
template<class Associated>
```

```
struct projected { };
```

with

```
template<class T>
struct __projected_impl {
  struct type { };
};

template<class NonAssociated>
using projected =
  __projected_impl<NonAssociated>::type;
```

ADL will associate the base classes of derived classes, but it does not associate the containing classes of nested classes. In short, the `::` in `__projected_impl<NonAssociated>::type` acts as a firewall against unwanted ADL.

For more information, see [Disable].

[P2300R4] actually provides blanket wording and a phrase of power that denotes this technique, in their proposed new library section [lib.tmpl-heads]. Quote:

> If a class template's *template-head* is marked with "arguments are not associated entities", any template arguments do not contribute to the associated entities of a function call where a specialization of the class template is an associated entity. In such a case, the class template may be implemented as an alias template referring to a templated class, or as a class template where the template arguments themselves are templated classes.

This is exactly the sort of thing I'm talking about, and if that wording is shipped, then we might consider rewording `std::projected` to use the phrase of power. However, I'm proposing a particular implementation technique below, so that the wording here (which I hope vendors will DR back to C++20) is not unnecessarily tied to the fate of P2300. The proposed wording here also removes some unnecessary complexity (a partial specialization of `incrementable_traits` that, after this patch, will not be physically possible to implement).

If we adopt this proposal and respecify `std::projected` along these lines, then the programmer will gain the ability to write:

```
using T = Holder<Incomplete>*;

static_assert(std::equality_comparable<T>); // OK
static_assert(std::indirectly_comparable<T*, T*, std::equal_to<>>); // will
static_assert(std::sortable<T*>); // will be OK

int main() {
  Holder<Incomplete> *a[10] = {}; // ten null pointers
  assert(std::count(a, a+10, nullptr) == 10); // OK
  assert(std::ranges::count(a, a+10, nullptr) == 10); // will be OK
}
```

§ 5. Implementation experience

This has been prototyped in a branch of libc++, and is just waiting for the paper to be adopted so that we can merge it. See [D119029].

§ 6. Proposed wording relative to N4868

> Note: The phrase of power "present only if..." was recently introduced into the working draft by [P2259R1]; see for example [counted.iterator] and [range.iota.iterator].

> Note: I believe this doesn't need a feature-test macro, because it merely enables code that was ill-formed before, and I can't think of a scenario where you'd want to do one thing if the feature was available and a different thing if it wasn't.

Modify [projected] as follows:

Class template `projected` is used to constrain algorithms that accept callable objects and projections. It combines ~~a~~ an `indirectly_readable` type `I` and a callable object type `Proj` into a new `indirectly_readable` type whose `reference` type is the result of applying `Proj` to the `iter_reference_t` of `I`.

```
~~namespace std {~~
~~  template<indirectly_readable I, indirectly_regular_unary_invocable<I> F~~
~~  struct projected {~~
~~    using value_type = remove_cvref_t<indirect_result_t<Proj&, I>>;~~
~~    indirect_result_t<Proj&, I> operator*() const; // not defined~~
~~  };~~

~~  template<weakly_incrementable I, class Proj>~~
~~  struct incrementable_traits<projected<I, Proj>> {~~
~~    using difference_type = iter_difference_t<I>;~~
~~  };~~
~~}~~

namespace std {
  template<class I, class Proj>
  struct projected-impl { // exposition only
    struct type { // exposition only
      using value_type = remove_cvref_t<indirect_result_t<Proj&, I>>;
      using difference_type = iter_difference_t<I>; // present only if I
      indirect_result_t<Proj&, I> operator*() const; // not defined
    };
  };

  template<indirectly_readable I, indirectly_regular_unary_invocable<I> F
  using projected = projected-impl<I, Proj>::type;
}
```

§ 7. Acknowledgments

- Thanks to Jonathan Wakely and Ville Voutilainen for recommending Arthur write this paper.
- Thanks to Barry Revzin for suggesting the "present only if..." wording, and to Hui Xie for pointing out the relevance of [P2300R4].

§ Appendix A: Compiler error messages for the `ranges::count` example

Here's the sample program again (Godbolt):

```
#include <algorithm>

template<class T> struct Holder { T t; };
struct Incomplete;

int main() {
    Holder<Incomplete> *a[10] = {}; // ten null pointers
    (void)std::count(a, a+10, nullptr); // OK
    (void)std::ranges::count(a, a+10, nullptr); // error
}
```

GCC (with libstdc++) says:

```
  In instantiation of 'struct Holder<Incomplete>':
bits/iterator_concepts.h:292:6:   required by substitution of 'template<cla
bits/stl_iterator_base_types.h:177:12:   required from 'struct std::iterato
bits/iterator_concepts.h:191:4:   required by substitution of 'template<cla
bits/iterator_concepts.h:204:13:   required by substitution of 'template<cl
bits/iterator_concepts.h:278:13:   required by substitution of 'template<cl
bits/iterator_concepts.h:283:11:   required by substitution of 'template<cl
bits/iterator_concepts.h:515:11:   required by substitution of 'template<cl
  required from here
error: 'Holder<T>::t' has incomplete type
    5 | template<class T> struct Holder { T t; };
      |                                       ^
note: forward declaration of 'struct Incomplete'
    4 | struct Incomplete;
      |        ^~~~~~~~~~
```

Clang (also with libstdc++, as libc++ does not yet implement `std::ranges::count`) says:

```
error: field has incomplete type 'Incomplete'
template<class T> struct Holder { T t; };
                                  ^
bits/iterator_concepts.h:292:6: note: in instantiation of template class 'H
          { *__it } -> __can_reference;
```

```
            ^
bits/iterator_concepts.h:292:6: note: in instantiation of requirement here
          { *__it } -> __can_reference;
            ^~~~~
bits/iterator_concepts.h:290:34: note: while substituting template argument
      concept __cpp17_iterator = requires(_Iter __it)
                                 ^~~~~~~~~~~~~~~~~~~
bits/iterator_concepts.h:298:40: note: while checking the satisfaction of c
      concept __cpp17_input_iterator = __cpp17_iterator<_Iter>
                                       ^~~~~~~~~~~~~~~~~~~~~~
bits/iterator_concepts.h:298:40: note: while substituting template argument
      concept __cpp17_input_iterator = __cpp17_iterator<_Iter>
                                       ^~~~~~~~~~~~~~~~~~~~~~
bits/iterator_concepts.h:388:11: note: (skipping 20 contexts in backtrace;
             && __detail::__cpp17_input_iterator<_Iterator>
                ^
bits/iterator_concepts.h:717:9: note: while substituting template arguments
      = indirectly_readable<_I1> && indirectly_readable<_I2>
        ^~~~~~~~~~~~~~~~~~~~~~~~
bits/ranges_algo.h:282:16: note: while checking the satisfaction of concept
      requires indirect_binary_predicate<ranges::equal_to,
               ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
bits/ranges_algo.h:282:16: note: while substituting template arguments into
      requires indirect_binary_predicate<ranges::equal_to,
               ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
note: while checking constraint satisfaction for template 'operator()<Holde
(void)std::ranges::count(a, a+10, nullptr); // hard error
                         ^
```

MSVC (with Microsoft STL, which generally does not pursue ADL-proofing anyway) says:

```
error C2079: 'Holder<Incomplete>::t' uses undefined struct 'Incomplete'
xutility(471): note: see reference to class template instantiation 'Holder<
xutility(485): note: see reference to variable template 'bool _Cpp17_iterat
xutility(362): note: see reference to class template instantiation 'std::it
xutility(417): note: see reference to variable template 'bool _Is_from_prim
xutility(718): note: see reference to alias template instantiation 'std::it
xutility(728): note: see reference to variable template 'bool _Indirectly_r
xutility(904): note: see reference to variable template 'bool indirectly_re
algorithm(466): note: see reference to variable template 'bool indirect_bin
```

§ References

§ Informative References

[D119029]

Arthur O'Dwyer. *[libc++] [D2358R0] ADL-proof `projected`*. February 2022. URL: https://reviews.llvm.org/D119029

[Disable]

Arthur O'Dwyer. *How hana::type<T> disables ADL*. April 2019. URL: https://quuxplusone.github.io/blog/2019/04/09/adl-insanity-round-2/

[P2259R1]

Tim Song. *Repairing input range adaptors and counted_iterator*. January 2021. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2259r1.html

[P2300R4]

Michał Dominiak; et al. *std::execution*. January 2022. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2300r4.html

[Uglification]

Arthur O'Dwyer. *ADL can interfere even with uglified names*. September 2019. URL: https://quuxplusone.github.io/blog/2019/09/26/uglification-doesnt-stop-adl/